

Project Definition

Following are text based project requirements for the Senior Project Management System.

Actors

- Coordinator
- Advisors
- Student

Roles

- Admin is a professor
- Coordinator is a professor
- Team Leader is a student

Grading

This section introduce components of the grading mechanism, describe how grading works with example calculations.

Grading Criteria

Binary Grading

- S - 100
- F - 0

Soft Grading

- A - 100
- B - 80
- C - 60
- D - 50
- F - 0

Evaluation Rubric

Collection of questions and a picker based on Soft Grading and Binary Grading.

Deliverables

Preset of deliverables are given below, deliverables can be configured by the coordinator.

- Proposal (Text Document)
- Statement of Work (Text Document)
- Demonstration

Team and Individual Evaluation

- Coordinators responsible for creating an evaluation rubric based with a grading criteria (binary/soft) for a Deliverable
- Setting per-sprint story point requirement for each student.
- Coordinator set which sprint contributes to which deliverable at what percentage.

Team Evaluation

- Point A: Advisor grade performance of the team each sprint using Soft Grading criteria.

- Point B: Advisor grade work/code review using Soft Grading criteria.

Average of Point A and B used as “coefficient of allowance” of team to take grade from the grade of a given deliverable.

An example scenario is given below.

Activity	Sprint 1	Sprint 2	Sprint 3	Total
Scrum	B	A	A	90
Work/Code Review	C	B	A	70

Presumed Deliverable Grades

Proposal	SoW	Demonstration
90	94	92

Associations (Set by Coordinator)

- Documents 50%
 - Sprint 1 → Proposal
 - Sprint 1, 2 → SoW
- Demonstration 50%
 - Sprint 1, 2, 3 → Demonstration

Given below describes how deliverable scalars calculated

	Proposal	SoW	Demonstration
Scrum	0,7	AVG(B(80), A(100))=90 → 0,9	AVG(B(80), A(100), A(100))=93 → 0,93
Reviews	0,6	AVG(C(60), B(80))=70 → 0,7	AVG(C(60), B(80), A(100))=80 → 0,8
AVG (Scrum, Reviews)	65 → 0,65	80 → 0,8	86,5 → 0,865

After Scalars Applied Scaled

Proposal	SoW	Demonstration
90 (proposal grade)*0,65 (proposal scalar) = 58,5	94*0,8=75,2	92*0,865=79,58

Grading Item/Deliverable Weights

- Documents 50%
 - Proposal, Weight: 15%
 - SoW, Weight: 35%
- Demonstration 50%
 - Presentation, Weight: 50%

Documents	Demonstration	Total
58,5x0,15=8,77 +75,2x0,35=26,32	79,58x0,5=74,55	74,88

Individual Evaluation

- Ratio of completing story points are used for evaluating individual contribution to deliverables and determine allowance of each student from the deliverable grade.

Student	Sprint 1 (Target 5)	Sprint 2 (Target 5)	Total (10)
11070001000	5	4	9

The student 11070001000 will get 0.9 of the overall grade that team takes, which is 71,622

Processes

Processes are describe the use-cases of each actor. However, affects other actors. Dependencies specified in the descriptions.

Setting Evaluation Rubric

- Coordinators responsible for creating an evaluation rubric based with a grading criteria (binary/soft) for a Deliverable
- Setting per-sprint story point requirement for each student.
- Coordinator set which sprint contributes to which deliverable at what percentage. (Story points are used for evaluating individual contribution to deliverables)

User Registration

Student

- Students register to system (valid student ids will be uploaded by the coordinator earlier)
- Students connect their GitHub accounts, their usernames will be fetched. (Features/ ' GitHub OAuth Integration')

Professor

- Will be manually registered by the admin
- They will request a password change on their initial login

Group Creation

- Bounded by a schedule set by the Coordinator
- A Student can create a group (which automatically appoint this student as the team leader of the first sprint)
- Team leader adds members, added students will be notified on the notifications section and they would have to approve, if they approve another request other awaiting requests will automatically be denied.
- Coordinator can manually remove or add a student from/to a group.
- Team leader will setup JIRA/Github integration (Features/ 'JIRA/GitHub Integration')

Group - Advisor Association

- Bounded by a schedule set by the Coordinator
- Team leader make an Advisee Request to a Professor or withdraw their request. Advisor will be notified in their notifications. If an advisor approves.
- An advisor accepted a group and they are matched, the advisor must release the team first in order them to make another advisee request.
- The coordinator can transfer a group from an advisor to another.

Sanitization

- Groups without an advisor will be disbanded.

Committee Assignment

- Requires “Group Creation” and “Group-Advisor Association” to be completed.
- Each advisor is a member of a committee.
- Each committee responsible for grading multiple groups.
- Advisor in a committee grade their own groups and groups of other advisors.
- Coordinator creates committee.
- Coordinator can also assign additional jury

Proposal Submission

- Bounded by a schedule set by the Coordinator
- Individual students cannot submit.
- Only groups assigned to a committee can submit therefore, requires “Committee Assignment” to be completed.
- Proposal

Proposal Review

- Bounded by a schedule set by the Coordinator
- Requires “Proposal Submission”.
- Committee members review and leave comments on the proposal.

Grading Reviewed Proposal Submission

- Bounded by a schedule set by the Coordinator
- Requires “Proposal Submission” if any changes requested in “Proposal Review”
- Committee members review and grade the proposal. According to grading criteria.

SoW Submission

- Bounded by a schedule set by the Coordinator
- The requires “Revised Proposal Submission” completed.
- Committee members review and grade the SoW. According to grading criteria.

Scrum

- Requires “Group - Advisor Association” completed. (So that)
- Scrum schedule will be set by the coordinator.
- Using `Features/'JIRA/GitHubIntegration'`, at the end of the sprint system refresh active stories in the sprint and verify story point estimates by validating wrt GitHub and register.
- Show live grades in advisor panel.

Features

GitHub OAuth Integration (Base)

- Request students to login to personal GitHub.
Students should be able to authenticate to GitHub easily with  [NextAuth.js](#) or similar OAuth framework. Their usernames are required for JIRA/GitHub Integration

JIRA/GitHub Integration (Base)

- Request teams to bind their JIRA spaces to their teams
- Request teams to bind their Github Organization with Personal Access Token

The system will fetch active stories in a sprint. (System should refresh daily by matching them by the issue key) (Issue Key, Work, Assignee, Reporter, Resolution, Created, Updated, Description). Then, should access github find branches

starting with the specified issue key and obtain related PR to check if it has been merged.

Admin password reset link (Difficulty Level 1)

Admin should generate a one-time-use password reset link.

Logging user events (Difficulty Level 1)

Trace logging user events.

Embedded Markdown Editor (Difficulty Level 2)

Markdown Editing with WYSIWYG Support

- Ability to insert images.
- Grading Criteria descriptions in Evaluation Rubric should be associated with a section in the markdown document.

AI to Read Pull Requests (Difficulty Level 3)

An AI will read pull request comments and verify that a review process has happened.

- Requires changes in 'Grading/Grading Criteria/Team Evaluation'

AI to Validate Issue Implementation (Difficulty Level 4)

AI to read file diffs introduced in PR and check wrt to the issue description.